

S2S-07: Dashboard, Workflow, and Integration Migration

Series: S2S | **Notebook:** 7 of 12 | **Created:** March 2026 | **Last Updated:** 03/23/2026

Overview

Dashboards, workflows, notification integrations, and synthetic monitors are the most user-visible configuration. Migrating them requires careful attention to entity ID references, webhook URLs, and ownership assignments.

Table of Contents

1. [Dashboard Migration](#)
 2. [Workflow Migration](#)
 3. [Notification Integration Migration](#)
 4. [Synthetic Monitor Migration](#)
 5. [Extension Migration](#)
-

Prerequisites

Requirement	Details
Target Tenant	IAM, enrichment, and buckets configured (S2S-03, S2S-05)
OAuth Client	<code>document:documents:write</code> , <code>automation:workflows:write</code> scopes
API Token	Classic token for Synthetic monitors (v1.88.0+ requirement)

1. Dashboard Migration

Classic Dashboards (Gen2)

Classic dashboards use the Dashboard API v1 and are exported as JSON:

```
# Export classic dashboards via Monaco
monaco download manifest.yaml --environment source --specific-api dashboard

# Or via Terraform
terraform import dynatrace_dashboard.my_dashboard "<dashboard-id>"
```

Note: Classic dashboards frequently contain hardcoded entity IDs in tile filters, management zone references, and metric selectors. These must be updated before importing to the target tenant.

Gen3 Documents (Dashboards & Notebooks)

Export and Import

```
# Export dashboards via Monaco
monaco download manifest.yaml --environment source --only-documents

# Or export via Terraform
terraform import dynatrace_document.my_dashboard "<dashboard-id>"
```

Entity ID Remediation in Dashboards

Dashboard JSON often contains hardcoded entity IDs. Before importing to the target:

Pattern to Find	Replace With
<code>entityId("HOST-xxxx")</code>	Tag-based filter or lookup new ID
<code>entityId("SERVICE-xxxx")</code>	<code>type(SERVICE), tag(app:name)</code>
<code>dt.entity.host == "HOST-xxxx"</code>	<code>dt.entity.host == "HOST-newid"</code>

Ownership and Sharing

Setting	Source	Target Action
Owner	Source user email	Reassign to target user or service account
Shared with	Specific users/groups	Update to target groups
Public	true/false	Maintain setting

2. Workflow Migration

Workflows are Gen3 resources requiring OAuth credentials:

```
# Export from source, apply to target
resource "dynatrace_automation_workflow" "problem_notify" {
  title      = "Problem Notification"
  description = "Notify Teams on critical problems"
  actor      = var.automation_service_user_id # New service user in target

  trigger {
    event {
      active = true
      config {
        davis_problem {
          categories {
            availability = true
            error        = true
          }
        }
      }
    }
  }
}

# ... tasks
}
```

Workflow Migration Checklist

Item	Action
Trigger configuration	Verify entity tags exist in target
Task actions	Update webhook URLs, API endpoints
Actor (service user)	Create new service user in target
Credentials	Create new connection credentials in target
Scheduled workflows	Verify cron expressions match desired timezone

3. Notification Integration Migration

Webhook URLs

Most notification integrations use webhook URLs that are not tenant-specific:

Integration	URL Changes?	Action
Microsoft Teams	No	Same webhook URL works
Slack	No	Same webhook URL works
PagerDuty	No	Same integration key works
ServiceNow	No	Same instance URL works
Jira	No	Same project URL works
Email	No	Same SMTP config
Custom Webhook	Depends	Verify endpoint accepts traffic from target tenant IP range

Notification Rules

Notification rules must be recreated in the target tenant with updated references:

```
resource "dynatrace_webhook_notification" "teams_critical" {
  active   = true
  name     = "Teams - Critical Alerts"
  profile  = dynatrace_alerting.critical.id # Reference to target alerting rule
  url      = var.teams_webhook_url          # Same URL, different variable source
  # ... payload
}
```

4. Synthetic Monitor Migration

Key Constraints

- Synthetic monitors require a **classic API token** (v1.88.0+)
- Synthetic ActiveGate locations are tenant-specific — recreate in target
- Public locations are shared and work across tenants

Migration Steps

Step	Action	Notes
1	Export Synthetic configs from source	Terraform or Monaco
2	Create Synthetic ActiveGate locations in target	Install AG with Synthetic capability
3	Update location references in config	Replace source location IDs with target IDs
4	Import to target tenant	Terraform apply (requires API token, not OAuth)
5	Verify execution	Check Synthetic results in target

5. Extension Migration

Extensions 2.0 must be reinstalled in the target tenant:

Step	Action	Notes
1	List extensions in source	Extensions API
2	Download extension packages	Or obtain from Dynatrace Hub
3	Upload to target tenant	Extensions API or UI
4	Configure extension instances	Settings API or Monaco
5	Activate monitoring	Verify data collection

Note: Extensions 2.0 require a host-based ActiveGate (not K8s-based). Ensure the target tenant has an appropriate ActiveGate deployment.

Next Steps

Continue to **S2S-08: OpenPipeline and Grail Bucket Migration** to migrate data routing configuration.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.